CRYPTOGRAPHY & NETWORK SECURITY (CNS) PROJECT PRESENTATION

KeyShield: Secure Password Manager

A Cryptographically Secure Local Credential Vault Using PBKDF2 & Fernet

Guide: Amey Muchandi

Team Members — Team B17	USN
S Srija	02FE23BCS048
Pranjali Mutagekar	02FE23BCS151
Prakhar Shreshta	02FE23BCS090
Harshit Baliyan	02FE23BCS092

Slide 2: Introduction & Problem Statement

The Password Security Dilemma

- **Cognitive Overload:** Modern users maintain 80–120+ digital accounts, rendering memorization mathematically impractical.
- **Credential Reuse:** Over 65% of individuals reuse identical passwords across sensitive email, corporate, and financial portals.
- **Attack Vectors:** Compromised or weak credentials account for over 80% of corporate data intrusions globally.
- **Insecure Storage Habits:** Plaintext spreadsheets, web-browser default caches, and written notes represent severe security liabilities.

Why Cloud Password Managers Fall Short

- **Centralized Target:** Cloud vaults concentrate millions of encrypted databases on remote infrastructure, presenting an attractive attack vector (e.g., LastPass breach).
- **Network Dependencies:** Reliance on third-party servers introduces supply chain vulnerabilities, API credential scraping, and MITM interception risks.

Problem Statement

There is an urgent necessity for a **zero-cloud, air-gapped, cryptographically authenticated credential vault** that ensures absolute confidentiality and integrity of stored credentials without exposing keys or metadata to remote networks.

Security Defense Model

Attack Vector	Conventional Vulnerability	KeyShield Defense
Rainbow Tables	Precomputed hash lookup attacks	16-byte random cryptographic salt
Brute-Force Cracking	Fast GPU wordlist iteration	PBKDF2-HMAC-SHA256 (200k rounds)
Ciphertext Tampering	Bit-flipping in unauthenticated AES	HMAC-SHA256 signature verification
SQL Injection	Direct database extraction	Parameterized queries (?) throughout

Slide 3: Project Objectives & Intended Features

Primary Technical Goals

1. **Zero Plaintext Persistence:** Ensure no sensitive credential or encryption key is ever stored in plaintext on disk.
2. **Cryptographic Key Derivation:** Implement NIST SP 800-132 PBKDF2-HMAC-SHA256 with 200,000 iterations to stretch the master password into a 256-bit symmetric key.
3. **Authenticated Symmetric Encryption:** Secure credentials using Fernet (AES-128-CBC + HMAC-SHA256) ensuring confidentiality and tamper resistance.
4. **CSPRNG Password Generation:** Deliver an entropy-guaranteed random password generator using Python's kernel-backed secrets module.
5. **Safe Database Transactions:** Prevent SQL injection vectors through parameterized query bindings across all SQLite CRUD operations.
6. **Ephemeral Session State:** Confine derived cryptographic keys strictly to volatile memory, discarding them completely upon user logout or exit.

Implementation Specifications

Component / Feature	Design Standard / Metric
Master Password Auth	Salted SHA-256 hash stored in data/master.hash
KDF Work Factor	200,000 rounds of PBKDF2-HMAC-SHA256 (~0.3s)
Encryption Scheme	Fernet authenticated envelope (AES-128 + HMAC)
Initialization Vector	16-byte cryptographically random IV per entry
Entropy Source	Python secrets module (OS CSPRNG pool)
Database Engine	SQLite with WAL (Write-Ahead Logging) mode
Test Coverage	49 automated tests spanning all functional modules

Designed specifically as an auditable academic project for Cryptography & Network Security.

Slide 4: Literature Survey

Review of 5–6 research papers on password managers, encryption algorithms, and credential security.

Ref.	Research Paper Title / Authors / Conference / Year	Key Findings & Relevance to KeyShield
[1]		
[2]		
[3]		
[4]		
[5]		

[This slide is reserved for your Literature Survey. You can write or paste your research papers directly into this template.]

Slide 5: Research Gap & Proposed Solution

Identified Industry & Research Gaps

- **Cloud Exfiltration Vulnerabilities:** Cloud credential vaults aggregate multi-user vaults into central servers, exposing users to systemic server-side breaches.
- **Absence of Key Stretching:** Naive local tools execute single-round cryptographic hashes (raw SHA-256/MD5), making them trivial to crack with modern GPUs.
- **Unauthenticated Encryption:** Basic AES in ECB/CBC mode without HMAC allows bit-flipping attacks where attackers manipulate stored ciphertext.
- **Weak Pseudo-Randomness:** Many generators use standard PRNGs (e.g. random.choice) instead of cryptographically secure random sources.

How KeyShield Addresses the Gaps

- **100% Offline Air-Gapped Operation:** Zero network listeners, zero telemetry, zero third-party cloud dependence.
- **200,000 Iteration PBKDF2 Stretching:** Forces attackers to incur a massive computational penalty per password attempt, defeating dictionary rigs.
- **Authenticated Fernet Token Envelope:** Authenticates ciphertext with HMAC-SHA256, instantly detecting any record modification or tampering.
- **OS-Seeded CSPRNG (secrets module):** Generates high-entropy passwords using system-level cryptographic randomness.

Capability / Feature	Browser Storage	Commercial Cloud Vault	KeyShield (Proposed)
Network Exposure	High (Browser ecosystem)	High (Remote Cloud)	**Zero (Air-Gapped Local)**
Key Derivation (KDF)	OS dependent / Weak	PBKDF2 / Argon2	**PBKDF2 (200k rounds)**
Integrity Check (MAC)	Absent	Present	**HMAC-SHA256 (Fernet)**
SQL Injection Defense	N/A	Proprietary backend	**Strict Parameterized (?)**
Offline Verifiability	Closed source	Proprietary	**100% Auditable Python**

Slide 6: Proposed System Architecture & Workflow

Operational System Workflow

1. Authentication Phase: User inputs master password → system checks data/master.hash. If valid, Session Key is derived using PBKDF2 and data/salt.bin.

2. Encryption Pipeline: Plaintext credential + derived Session Key → Fernet generates random 16-byte IV → AES-128-CBC encryption → HMAC-SHA256 signature generated.

3. Secure Storage: Ciphertext token is committed to SQLite (vault.db) as a binary BLOB via parameterized queries.

4. Decryption Pipeline: Binary BLOB retrieved → HMAC signature validated → AES decrypted → Rendered into GUI memory.

5. Session Teardown: On logout, Session Key in memory is set to None and garbage collected.

Component Modularization

Component	Module	Cryptographic / System Responsibility
Application Core	main.py	Database bootstrap, CustomTkinter lifecycle
Auth Engine	auth.py	Salted SHA-256 hash creation and verification
Crypto Engine	encryption.py	PBKDF2-HMAC-SHA256 & Fernet encrypt/decrypt
Data Layer	db_manager.py	Parameterized SQLite CRUD with BLOB storage
Generator	password_gen.py	CSPRNG-driven random password generation
GUI Controller	gui.py	CustomTkinter dashboard & live strength meter

Database Schema:

```
credentials(id PK, website TEXT, username TEXT, encrypted_pass BLOB, notes TEXT, created_at, updated_at)
```

Slide 7: Methodology & Technologies

Cryptographic Primitives In-Depth

1. PBKDF2-HMAC-SHA256 (Key Derivation):
Formula: DK = PBKDF2(HMAC-SHA256, Password, Salt, 200000, 32)
Produces 32-byte key (base64-encoded to 44 bytes). The 200,000 iterations require ~0.3s per run, rendering brute force dictionary attacks computationally prohibitive.

2. Fernet Symmetric Authenticated Encryption:
Combines AES-128-CBC for confidentiality with HMAC-SHA256 for integrity verification. Token structure: Version || Timestamp || IV(16B) || Ciphertext || HMAC(32B).

3. Cryptographic Salt Management:
16-byte random salt generated via os.urandom(16) on first run and stored in data/salt.bin.

4. CSPRNG Random Generation:
Uses Python's secrets module (backed by kernel entropy) with character set guarantee logic.

Technology Stack & Verification

- **Python 3.12:** High-performance runtime with modern typing.
- **cryptography & hashlib:** Standards-compliant crypto primitives.
- **CustomTkinter 5.2:** Modern dark UI with responsive elements.
- **SQLite 3:** Serverless, zero-configuration local database.

Automated Test Suite (49/49 Passing)

Test Module	Pass	Verification Scope
TestEncryption	11 / 11	Roundtrip encrypt/decrypt, salt corruption
TestDatabase	10 / 10	CRUD operations, no-plaintext in DB check
TestAuth	9 / 9	Master setup, salted hash verification
TestPasswordGen	14 / 14	Entropy, length boundaries, character classes
TestIntegration	4 / 4	Full end-to-end credential management

Slide 8: Expected Outcomes, Conclusion & References

Delivered Outcomes

- **Complete Plaintext Elimination:** All passwords stored strictly as binary Fernet tokens; verified via database blob inspection.
- **Robust Local Security:** Zero remote network dependencies, delivering complete privacy from external eavesdroppers.
- **100% Test Pass Rate:** All 49 automated unit and integration tests successfully verified under Python 3.12.

Future Enhancements

- **Argon2id Key Derivation:** Adopt memory-hard Argon2id to resist dedicated ASIC/FPGA cracking hardware.
- **Clipboard Auto-Clear:** Automatically purge copied passwords from OS clipboard memory after 30 seconds.
- **Encrypted Vault Backup:** Provide portable encrypted export/import capabilities for disaster recovery.

Conclusion

KeyShield proves that production-grade credential security can be achieved locally using standardized cryptographic algorithms. By uniting **PBKDF2 key stretching (200k rounds)**, **Fernet authenticated encryption**, **salted SHA-256 authentication**, and **CSPRNG entropy generation**, the project thoroughly demonstrates the practical implementation of core CNS syllabus concepts.

Key References

[1] NIST SP 800-132: *Recommendation for Password-Based Key Derivation*, National Institute of Standards and Technology, 2010.

[2] NIST FIPS 197: *Advanced Encryption Standard (AES)*, Federal Information Processing Standards Publication, 2001.

[3] RFC 2104: *HMAC: Keyed-Hashing for Message Authentication*, Internet Engineering Task Force (IETF).

[4] OWASP: *Password Storage Cheat Sheet*, Open Web Application Security Project.

[5] Python Cryptographic Authority: *Fernet Symmetric Encryption Specification*.